نظام حجز رحلات البولمان بين المحافظات — الدراسة التحليلية

> المصدر: ملف "المشروع.pdf" (متطلبات صاحب الشركة). المكدّس التقني: Figma (تصميم) · Express.js + Next.js (Backend API) · MongoDB (لوحة الإدارة والاستقبال) · React Native (تطبيق الزبون).

---

1. ملخص المشكلة

شركة نقل بين المحافظات السورية تملك أسطول بولمانات بثلاث درجات وحجمين. المواطن لا يعرف الرحلات المتاحة ولا أسعارها ولا يستطيع الحجز بسهولة. المطلوب:

1. **نظام إدارة** لإنشاء الرحلات (يدوية ومجدولة تلقائياً).
2. **واجهة لموظف الاستقبال** بصلاحيات محدودة يحجز من خلالها للزبائن.
3. **تطبيق موبايل** يستعرض فيه الزبون الرحلات، يرى خريطة مقاعد البولمان (الفارغ والمحجوز)، ويحجز المقعد الذي يختاره.

---

2. تحليل أنواع البولمانات (Seat Layouts)

الدرجة	التوزيع	الحجم	الصفوف	المقاعد	التحقق
عادية	2+2	كبير	11	45	4×11 = 44 + مقعد الوسط بآخر الممر = 45 ✅
عادية	2+2	متوسط	9	37	4×9 = 36 + 1 = 37 ✅
درجة أولى	2+1	كبير	11	33	3×11 = 33 ✅
درجة أولى	2+1	متوسط	9	27	3×9 = 27 ✅
رجال أعمال	1+1	كبير	11	22	2×11 = 22 ✅
رجال أعمال	1+1	متوسط	9	18	2×9 = 18 ✅

الاستنتاج: لا داعي لرسم كل مخطط يدوياً ولا لتخزينه بقاعدة البيانات. المخطط يُولَّد بالكود من درجة الباص وحجمه:

```
layout = { leftSeats, rightSeats, rows, hasBackMiddleSeat }
REGULAR     → { 2, 2, rows, true  }
FIRST_CLASS → { 2, 1, rows, false }
BUSINESS    → { 1, 1, rows, false }
rows: LARGE = 11, MEDIUM = 9
```

ترقيم المقاعد: رقم تسلسلي (1..N) من الأمام للخلف ومن اليسار لليمين. دالة واحدة `buildSeatMap(class, size)` ترجع لكل مقعد رقمه وإحداثياته (`row`, `column`)، ويرسمه الموبايل والويب بنفس الشكل.

2.1 المحافظات (ثابتة)

المدن هي **المحافظات السورية الأربع عشرة**، معرّفة كقائمة ثابتة (enum) بالكود، بلا جدول ولا إضافة أو تعديل أو حذف:

> دمشق · ريف دمشق · حلب · حمص · حماة · اللاذقية · طرطوس · إدلب · دير الزور · الرقة · الحسكة · درعا · السويداء · القنيطرة

- الـ API يعرضها للقراءة فقط (`GET /api/cities`).
- قاعدة تحقق: المصدر ≠ الوجهة.

---

3. الأدوار والصلاحيات

النظام فيه **ثلاثة أدوار فقط**:

الدور	من هو	أين يعمل	الصلاحيات
Admin	صاحب الشركة / الإدارة	لوحة الويب (Next.js)	تحكم كامل: الباصات، الرحلات، الجداول، حسابات موظفي الاستقبال، كل الحجوزات، التقارير
Receptionist	موظف الاستقبال بالمكتب/الكراج	لوحة الويب (Next.js) بواجهة مختصرة	استعراض الرحلات، الحجز للزبون، تأكيد الدفع، إلغاء حجز، تبديل مقعد، قائمة الركاب. **لا** ينشئ أو يعدّل رحلات أو أسعاراً
Customer	الزبون	تطبيق الموبايل (React Native)	تسجيل، استعراض الرحلات، حجز، إلغاء ضمن السياسة، عرض حجوزاته

مصفوفة الصلاحيات:

العملية	Admin	Receptionist	Customer
إدارة الباصات	✅	❌	❌
إنشاء/تعديل/إلغاء رحلة	✅	❌	❌
إدارة الرحلات المجدولة	✅	❌	❌
إدارة حسابات موظفي الاستقبال	✅	❌	❌
استعراض الرحلات ومخطط المقاعد	✅	✅	✅
حجز لزبون (من المكتب)	✅	✅	❌
حجز لنفسه (من التطبيق)	❌	❌	✅
تأكيد الدفع (نقداً أو شام كاش)	✅	✅	❌
إلغاء حجز	✅ أي حجز	✅ أي حجز	✅ حجوزاته فقط، ضمن السياسة
قائمة ركاب الرحلة (Manifest)	✅	✅	❌
التقارير	✅	❌	❌

التنفيذ: حقل `role` على مستند المستخدم (`ADMIN` / `RECEPTIONIST` / `CUSTOMER`)، وMiddleware بسيط مثل `authorize('ADMIN', 'RECEPTIONIST')` على كل Route.

---

4. المتطلبات الوظيفية (Functional Requirements)

4.1 لوحة الإدارة — Admin (Next.js)

> المحافظات وأنواع البولمانات ثابتة بالكود ولا تُدار من اللوحة — انظر القسمين 2 و 2.1.

- **FR-01 إدارة الباصات:** إضافة/تعديل/تعطيل باص: رقم اللوحة، الدرجة (عادية / درجة أولى / رجال أعمال)، الحجم (كبير / متوسط)، الحالة (فعّال / صيانة). الدرجة والحجم يحددان مخطط المقاعد تلقائياً، مع معاينة المخطط.
- **FR-02 إدارة الرحلات:** إنشاء رحلة (المصدر، الوجهة، الباص، سعر المقعد، التاريخ، الساعة) + تعديلها أو إلغاؤها. لا يُسمح بتغيير الباص إذا كان في الرحلة حجوزات.
- **FR-03 الرحلات المجدولة**: قالب رحلة + تكرار (يومي / أسبوعي بأيام محددة / شهري بيوم محدد) + تاريخ بداية ونهاية → توليد تلقائي.
- **FR-04** إيقاف/تعديل جدول دون التأثير على الرحلات التي فيها حجوزات.
- **FR-05** إدارة حسابات موظفي الاستقبال: إنشاء، تعطيل، إعادة تعيين كلمة المرور.
- **FR-06** تقارير: إشغال الرحلات، الإيرادات حسب الخط/الفترة، عدد الحجوزات لكل موظف استقبال.

4.2 واجهة الاستقبال — Receptionist (Next.js)

- **FR-07** بحث سريع عن رحلة (من/إلى/تاريخ).
- **FR-08** عرض مخطط المقاعد وحجز مقعد أو أكثر لزبون (الاسم، الهاتف).
- **FR-09** تأكيد الدفع (استلام نقدي أو مطابقة رقم عملية شام كاش)، إلغاء، تبديل مقعد.
- **FR-10** قائمة ركاب الرحلة قبل الانطلاق.

4.3 تطبيق الزبون — Customer (React Native)

- **FR-11** تسجيل/دخول برقم الهاتف وكلمة المرور.
- **FR-12** البحث عن رحلات (من، إلى، تاريخ) وعرض النتائج (الساعة، الدرجة، السعر، المقاعد المتبقية).
- **FR-13** صفحة الرحلة: **مخطط البولمان** بألوان (متاح / محجوز / محدَّد).
- **FR-14** اختيار مقعد أو أكثر وإدخال أسماء الركاب، ثم اختيار طريقة الدفع (نقداً بالمكتب / شام كاش مع رقم العملية) والحجز.
- **FR-15** "حجوزاتي": الحالية والسابقة، مع رمز الحجز وأرقام المقاعد.
- **FR-16** إلغاء حجز ضمن سياسة الإلغاء.

> **خارج النطاق حالياً:** الإشعارات (Push/SMS)، رمز QR، وطباعة/تصدير التذاكر PDF.

---

5. المتطلبات غير الوظيفية (Non-Functional)

البند	المتطلب
منع الحجز المزدوج	لا يُحجز نفس المقعد مرتين حتى لو ضغط زبونان وموظف بنفس اللحظة (انظر 6.1)
اللغة والاتجاه	عربي RTL أساساً
التوقيت	كل الأوقات بتوقيت `Asia/Damascus`، تُخزّن UTC
الشبكة الضعيفة	تطبيق خفيف، رسائل خطأ واضحة
الأمان	JWT، كلمات مرور مشفّرة (bcrypt)، فحص الدور على كل Route، تحقق من المدخلات

---

6. نقاط حرجة وقرارات تصميم

6.1 منع الحجز المزدوج

- المقاعد المحجوزة تُحفظ داخل مستند الرحلة نفسه (`bookedSeats`).
- الحجز عملية واحدة في MongoDB تنجح فقط إذا كانت كل المقاعد المطلوبة فارغة، وإلا يرجع خطأ `409 Conflict`.
- لا يوجد حجز مؤقت: المقعد يُحجز لحظة تأكيد الحجز.

- عند الإلغاء تُزال المقاعد من `bookedSeats` فتعود متاحة.

6.2 توليد الرحلات المجدولة

- `tripschedules` يخزن القالب + قاعدة التكرار.
- Job يومي (node-cron) يبقي رحلات **شهر كامل** قادم جاهزة (30 يوماً)، فكل يوم يُضاف اليوم الجديد بآخر الشهر.
- لا تتكرر الرحلة: Unique index على (`scheduleId` , `departureAt`).

6.3 الرحلة والباص

- كل رحلة مرتبطة بباص محدد، ومن درجته وحجمه يُعرف مخطط المقاعد.

6.4 الدفع

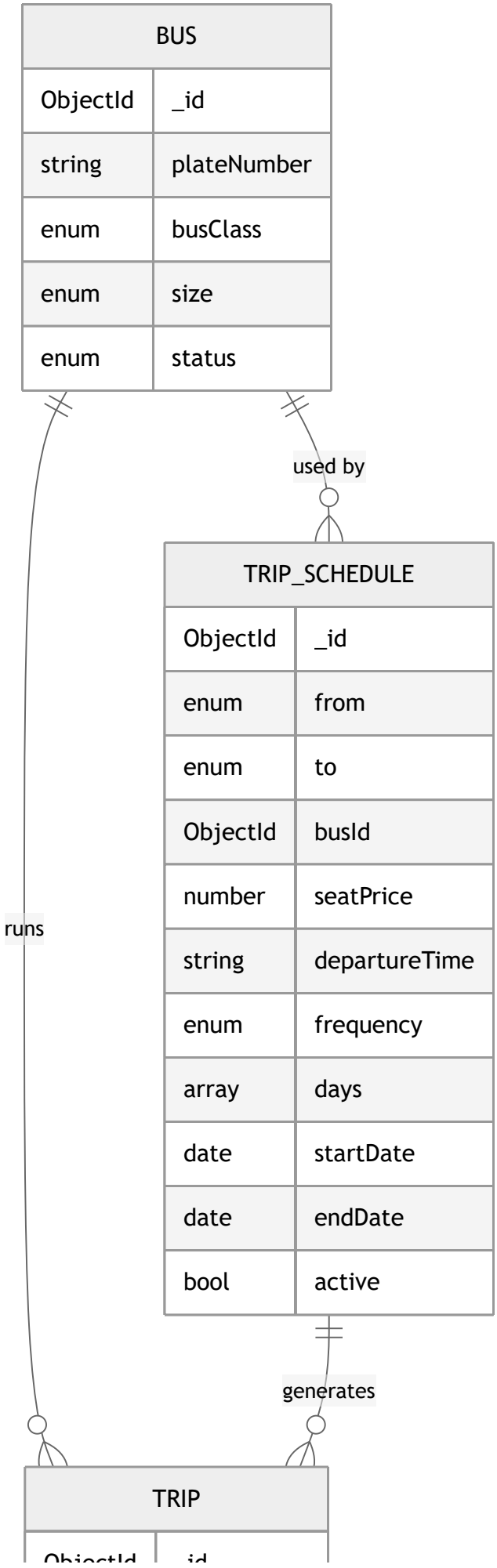
- طريقتان: **نقداً** بالمكتب، أو **شام كاش** بتحويل لحساب الشركة.
- حجز التطبيق يبدأ `PENDING`. مع شام كاش يُدخل الزبون رقم عملية التحويل (`paymentRef`).
- موظف الاستقبال يستلم المبلغ أو يطابق رقم العملية بحساب الشركة، فيصبح الحجز `CONFIRMED`.

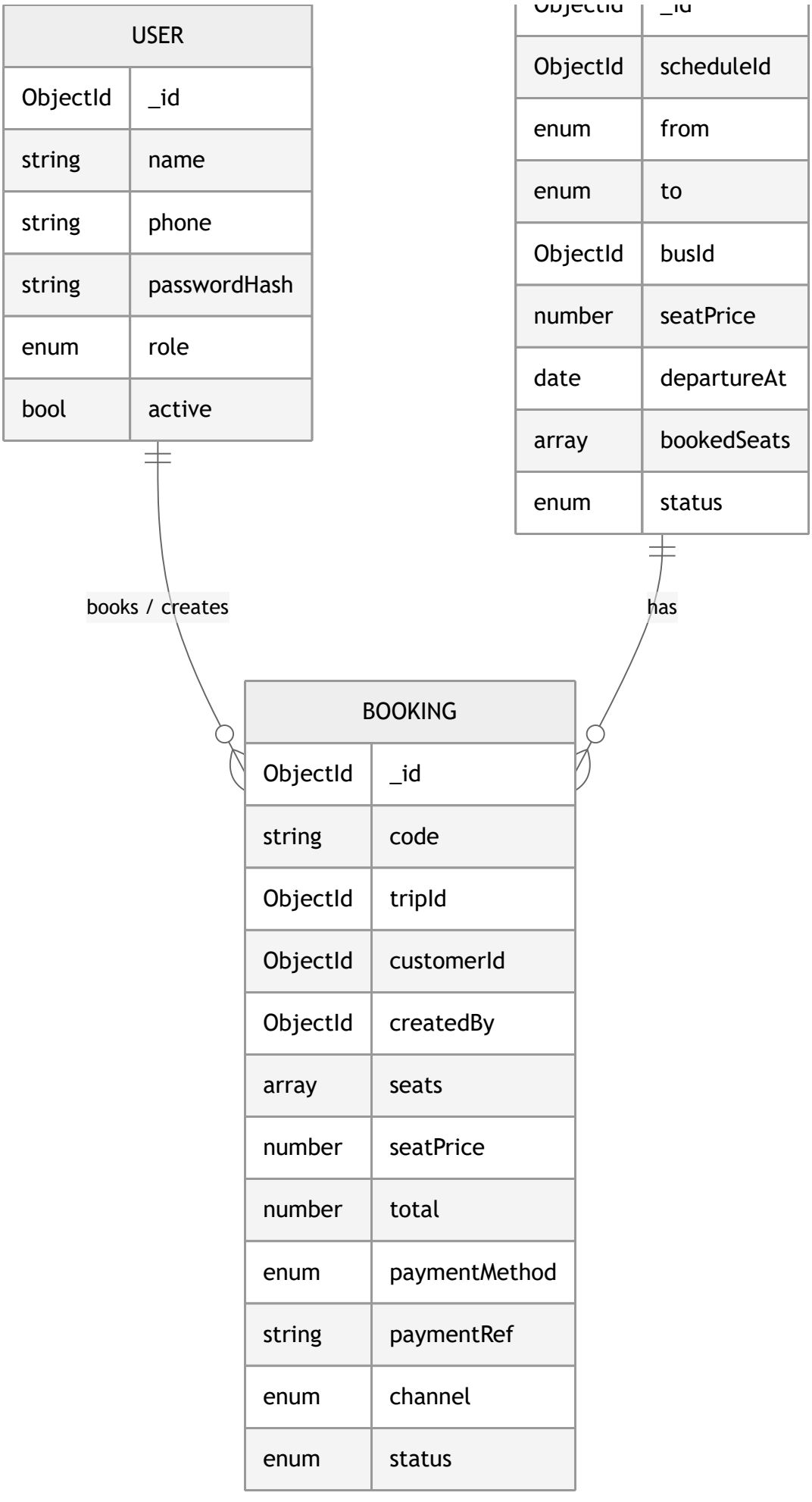
6.5 تثبيت السعر

- سعر المقعد يُنسخ إلى الحجز لحظة الحجز (`seatPrice`) حتى لا يتأثر بتعديل سعر الرحلة لاحقاً.

---

7. نموذج البيانات (MongoDB — 5 Collections)


BUS
ObjectId
_id
string
plateNumber
enum
busClass
enum
size
enum
status
used by
TRIP_SCHEDULE
ObjectId
_id
enum
from
enum
to
ObjectId
busId
number
seatPrice
string
departureTime
enum
frequency
array
days
date
startDate
date
endDate
bool
active
runs
generates
TRIP


USER
ObjectId
_id
string
name
string
phone
string
passwordHash
enum
role
bool
active
ObjectId
scheduleId
enum
from
enum
to
ObjectId
busId
number
seatPrice
date
departureAt
array
bookedSeats
enum
status
books / creates
has
BOOKING
ObjectId
_id
string
code
ObjectId
tripId
ObjectId
customerId
ObjectId
createdBy
array
seats
number
seatPrice
number
total
enum
paymentMethod
string
paymentRef
enum
channel
enum
status

- `BOOKING.seats` مصفوفة مضمّنة: `[{ seatNumber, passengerName }]` — بدل جدول منفصل للمقاعد.
- `from` / `to` من قائمة المحافظات الثابتة، و`busClass` / `size` من أنواع البولمانات الثابتة.

الأدوار (`role`): `ADMIN` / `RECEPTIONIST` / `CUSTOMER` **حالات الحجز:** `PENDING → CONFIRMED → COMPLETED` أو `CANCELLED` **حالات الرحلة:** `SCHEDULED → DEPARTED → ARRIVED` أو `CANCELLED` **قناة الحجز (`channel`):** `APP` / `RECEPTION` — مفيدة للتقارير. **طريقة الدفع (`paymentMethod`):** `CASH` / `SHAM_CASH`، و`paymentRef` رقم عملية شام كاش.

---

8. تصميم الـ API (مسودة)

```
# Auth
POST   /api/auth/register               (Customer)
POST   /api/auth/login                  (All roles)

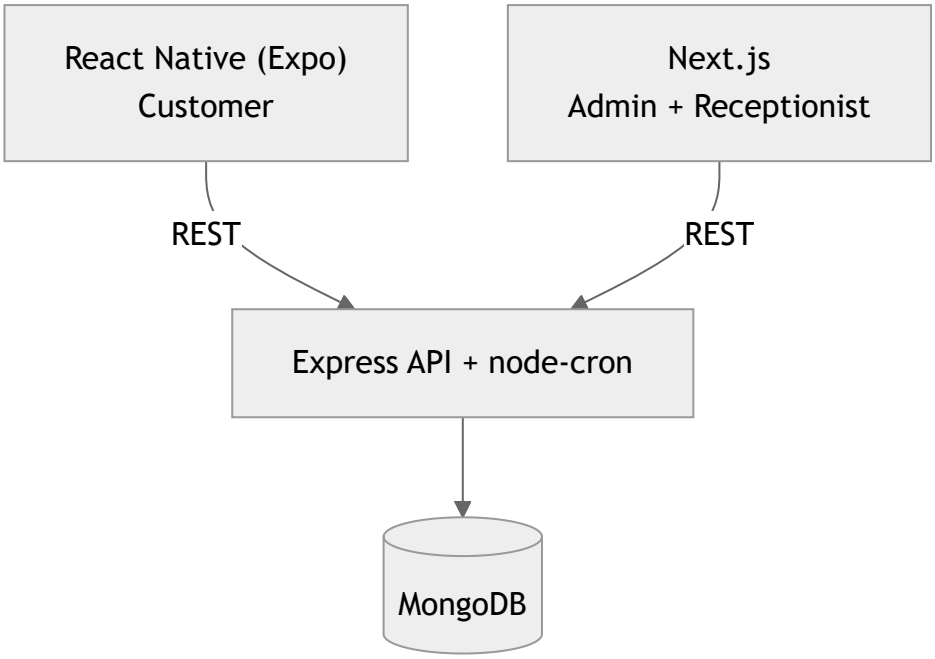
# Public + Customer
GET    /api/cities                      (المحافظات الثابتة — قراءة فقط)
GET    /api/trips?from=&to=&date=&class=
GET    /api/trips/:id                   (التفاصيل + seat map + المقاعد المحجوزة)
POST   /api/bookings                    { tripId, seats, paymentMethod, paymentRef? }
GET    /api/me/bookings
POST   /api/bookings/:id/cancel

# Admin + Receptionist
GET    /api/staff/trips?date=
POST   /api/staff/bookings              (حجز من الاستقبال)
PATCH  /api/staff/bookings/:id/confirm-payment
PATCH  /api/staff/bookings/:id/change-seat
POST   /api/staff/bookings/:id/cancel
GET    /api/staff/trips/:id/manifest    (قائمة الركاب)

# Admin only
CRUD   /api/admin/buses | receptionists
CRUD   /api/admin/trips                 POST /api/admin/trips/:id/cancel
CRUD   /api/admin/schedules
GET    /api/admin/reports/occupancy | revenue
```

---

9. المعمارية والتقنيات



الطبقة	الاختيار	السبب
Backend	Express.js	طلبكم
قاعدة البيانات	**MongoDB** + Mongoose	مرنة، والمصفوفات المضمّنة تبسّط الحجز (`seats` , `bookedSeats`)
Jobs	node-cron	توليد الرحلات المجدولة
Auth	JWT + bcrypt	
Web	Next.js + Tailwind	واجهة واحدة، القائمة الجانبية تتغير حسب الدور
Mobile	React Native (Expo)	

هيكل المستودع: مجلد لكل جزء — `backend/` · `web/` · `mobile/`.

---

10. شاشات Figma المطلوبة

Design System أولاً: ألوان، خطوط عربية (مثلاً Cairo / IBM Plex Sans Arabic)، أزرار، حقول، بطاقات، **مكوّن المقعد بحالاته** (متاح، محجوز، محدد، مقعد السائق/الباب)، مكوّن مخطط البولمان للأنواع الثلاثة.

الموبايل — Customer:

1. Splash / Onboarding
2. تسجيل الدخول / إنشاء حساب
3. الرئيسية / البحث (من، إلى، تاريخ)
4. نتائج البحث

5. تفاصيل الرحلة + مخطط المقاعد
6. أسماء الركاب
7. ملخص الحجز + اختيار طريقة الدفع (نقداً / شام كاش)
8. نجاح الحجز (رمز الحجز + المقاعد)
9. حجوزاتي (قادمة / سابقة)
10. تفاصيل الحجز + إلغاء
11. الملف الشخصي

الويب — مشترك (Admin + Receptionist):

1. تسجيل الدخول
2. **شاشة الحجز:** بحث سريع + مخطط مقاعد + نموذج حجز سريع
3. الحجوزات · تفاصيل حجز
4. قائمة ركاب الرحلة (Manifest)

الويب — Admin فقط: 5. Dashboard (رحلات اليوم، الإشغال، الإيرادات) 6. الباصات: قائمة + إضافة/تعديل (الدرجة والحجم مع معاينة المخطط) 7. الرحلات: جدول + فلترة · إنشاء/تعديل/إلغاء رحلة 8. الجداول المجدولة · إنشاء جدول 9. موظفو الاستقبال (إضافة، تعطيل، إعادة تعيين كلمة المرور) 10. التقارير

---

11. مراحل التنفيذ المقترحة

المرحلة	المحتوى
0. التحضير	الإجابة عن الأسئلة المفتوحة، Figma Design System + الشاشات
1. الأساس	Auth + الأدوار الثلاثة، الثوابت (المحافظات وأنواع البولمانات) ومولّد المخطط، إدارة الباصات
2. الرحلات	CRUD الرحلات، الجداول + Job التوليد
3. الحجز	الحجز الذرّي، الإلغاء، تأكيد الدفع، قائمة الركاب
4. الواجهات	لوحة Admin وشاشة Receptionist (Next.js)، تطبيق Customer (React Native)
5. الإكمال	التقارير، نشر تجريبي، بيانات تجريبية، README

---

12. أسئلة مفتوحة لصاحب الشركة

1. **فصل الجنسين بالمقاعد:** هل يُسمح بجلوس امرأة بجانب رجل غريب؟ (عُرف شائع بسوريا ويؤثر على منطق اختيار المقعد.)
2. **مهلة الدفع:** ماذا يحدث لحجز تطبيق لم يُدفع قبل الرحلة؟
3. **سياسة الإلغاء:** حتى متى يمكن الإلغاء؟ هل يوجد خصم/غرامة؟
4. **المحطات الوسطية:** هل الرحلة دمشق ← حلب تتوقف بحمص وحماة ويمكن الحجز لجزء من الطريق؟ (يغيّر نموذج البيانات كثيراً.)

5. **الأطفال:** هل للطفل مقعد مستقل وسعر مختلف؟
6. **عدد المقاعد للحجز الواحد:** هل يوجد حد أقصى؟
7. **مقاعد محجوزة للمكتب:** هل تُحجز مقاعد للبيع بالكراج فقط؟
8. **الأسعار:** ثابتة للخط والدرجة أم تختلف حسب الرحلة/الموسم؟
9. **الرحلات الشهرية:** ماذا يحدث إذا كان اليوم 31 والشهر أقصر؟
10. **الفروع:** هل للشركة أكثر من مكتب استقبال (موظف مرتبط بمدينة معينة)؟

---

13. الافتراضات الحالية (حتى تُجاب الأسئلة)

- ثلاثة أدوار فقط: Admin، Receptionist، Customer.
- خمس Collections فقط: `users` · `buses` · `trips` · `tripschedules` · `bookings`.
- المحافظات الأربع عشرة وأنواع البولمانات الستة ثابتة بالكود بلا إدارة.
- لا حجز مؤقت، ولا إشعارات، ولا QR، ولا تذاكر PDF؛ الزبون يعرض رمز الحجز من التطبيق.
- رحلة مباشرة من مدينة لمدينة بلا محطات حجز وسطية.
- سعر واحد لكل المقاعد في الرحلة.
- الدفع نقداً بالمكتب أو بتحويل شام كاش يتحقق منه موظف الاستقبال يدوياً (بلا ربط برمجي مع شام كاش).
- لا قيود جنس على المقاعد.